

Reviewer Pack — Tropical Rank Gap in Read-Twice BP Transition Matrices

Entry 965a23f70cf0 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-10 20:25:52 UTC

Tropical Rank Gap in Read-Twice BP Transition Matrices

Entry ID: 965a23f70cf0 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-10 20:25:52 UTC

1. Conjecture

Field A (mathematical branch): Tropical Linear Algebra **Field B** (complexity object): Read-Twice Branching Programs

Statement:

For any read-twice BP over $\text{GF}(2)$ with n variables, the tropical rank of its transition matrix is $\Omega(n)$, while read-once BPs have tropical rank $O(\log n)$.

Rationale (proposer’s reasoning):

Tropical rank captures the minimal number of rank-1 tropical matrices needed to decompose a matrix, reflecting structural complexity. Read-twice BPs, with their repeated state access, induce higher tropical rank due to entangled state transitions, contrasting with read-once BPs’ decomposable structure.

Taxonomy category: BP_READTWICE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): dd7ff9dda18f23a7

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from collections import defaultdict

def generate_bp_transition_matrix(clauses, n, p):
    A = [[0] * (2**n) for _ in range(2**n)]
    for s in range(2**n):
        for t in range(2**n):
            if any((s >> (v-1) & 1) == (t >> (abs(v)-1) & 1)) or all((s >> (v-1) & 1) != (t >> (abs(v)-1) & 1)) for v in clauses):
                A[s][t] = 1
    return A

def tropical_rank(A):
    m, n = len(A), len(A[0])
    rank = 0
    for i in range(m):
        if any(A[i][j] != 0 for j in range(n)):
            rank += 1
            for j in range(n):
                if A[i][j] != 0:
                    for k in range(m):
                        A[k][j] = min(A[k][j], A[k][i] + A[i][j])
    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 40
    p = 0.5
    instances_tested = 30
    total_rank = 0
    conjecture_holds = True
    counterexample = ""

    for _ in range(instances_tested):
        clauses = [[random.randint(1, n), random.choice([-1, 1]), random.randint(1, n), random.choice([-1, 1])] for _ in range(10)]
        A = generate_bp_transition_matrix(clauses, n, p)
        rank = tropical_rank(A)
        total_rank += rank
        if rank < 0.3 * n:
            conjecture_holds = False
            counterexample = f"Read-twice BP with rank {rank} < 0.3n"

    return {
        "metric_name": "tropical_rank",
        "metric_value": total_rank / instances_tested,
```

```

        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 35)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    std_rank = math.sqrt(sum((r["metric_value"] - mean_rank)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_rank:.4f} std={std_rank:.4f} support_fraction={support_fraction:.4f}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rank:.4f} std={std_rank:.4f} support_fraction={support_fraction:.4f}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={result['counterexample']} first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5231202d.py", line 157, in <module>
    result = run_trial(seed)
              ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5231202d.py", line 131, in run_trial
    A = generate_bp_transition_matrix(clauses, n, p)
        ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5231202d.py", line 110, in generate_bp_
    if any(all((s >> (v-1) & 1) == (t >> (abs(v)-1) & 1)) or all((s >> (v-
1) & 1) != (t >> (abs(v)-1) & 1)) for v in range(1, n+1)):
    ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5231202d.py", line 110, in <genexpr>
    if any(all((s >> (v-1) & 1) == (t >> (abs(v)-1) & 1)) or all((s >> (v-
1) & 1) != (t >> (abs(v)-1) & 1)) for v in range(1, n+1)):
    ~~~~~^~~~~~
TypeError: 'bool' object is not iterable

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to `TypeError`, preventing data collection. Pre-registered support condition cannot be evaluated without successful trials. | next: Debug the `generate_bp_transition_matrix` function's boolean expression handling

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	111318
2	propose	ollama_remote	qwen3:8b	0	0	33088
3	propose	ollama_remote	qwen3:8b	0	0	80537
4	preregistration	ollama_remote	qwen3:8b	0	0	24209
5	novelty	ollama_remote	qwen3:8b	0	0	20639
6	novelty	ollama_remote	qwen3:8b	0	0	18116
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19309
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9254
9	judge	ollama_remote	qwen3:8b	0	0	17583

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 334054 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/965a23f70cf0.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/965a23f70cf0.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/965a23f70cf0.tar.gz` (if generated)